

Plan: Accelerated Branch-and-Bound (ABB) for Global Top- K Cycle Enumeration

2026-05-10

Scope and goal

Add a profile-grounded score upper-bound branch-and-bound (ABB) descent to `hymeko_graph::topk_cycles`'s *global* top- K enumerator (`enumerate_top_k_cycles + _par` variant), behind a new opt-in entry point that takes a `BoundedScorer` trait object instead of a plain `Fn(&[u32], &[i8]) -> f64`.

The existing `enumerate_top_k_cycles_par` and `_per_vertex_cycles_par` entry points stay untouched; ABB is strictly additive.

Profile evidence (probe at `hymeko_graph/examples/probe_abb_threshold.rs`): on Epinions $k=4$ + `CartwrightHararyPruner(OnlyBalanced)` + `fraction_negative` scorer, 22.0% of balanced 4-cycles are *all-negative* (score 1.0). The global top- K heap threshold settles at $T=1.0$ for any $K \leq 1\,397\,554$. At $T=1.0$ the upper-bound rule $UB(d, n_{\text{neg}}) = (n_{\text{neg}} + (k - d))/k$ prunes *every* partial path where any committed edge is positive ($n_{\text{neg}} < d$). Estimated gain: 5–10 $\times$ wall-time reduction on the global top- K Epinions workload.

Out of scope: the per-vertex variant (`enumerate_top_k_per_vertex_cycles_par`). Companion probe `probe_per_vertex_thresholds.rs` confirmed only 18% of Epinions vertices fill their $m=128$ heap; $P(\text{all 4 cycle vertices full}) \approx 0.1\%$ uniform-bound, so per-vertex ABB rarely fires without a degree-adaptive m_v change that would also affect HSiKAN training distributions (separate plan, requires n -seed paired AUC validation).

Affected files

- `hymeko_graph/src/topk_cycles.rs` — additive:
 - New public trait `BoundedScorer` (`Sync + Send`) with `score(&self, vs, signs) -> f64` and `upper_bound(&self, n_neg_so_far, k_remaining, k_len) -> f64`.
 - Concrete implementations: `FractionNegativeScorer`, `BalanceScorer`, `SignProductAbsScorer`, `LowRootScorer` — mirror of the existing `scorers::` module, augmented with admissible UBs.
 - New entry points: `enumerate_top_k_cycles_bb<P, S>(graph, k, pruner, k_keep, scorer)` and `enumerate_top_k_cycles_par_bb<P, S>(...)` where `S: BoundedScorer`. Same `TopKCycle` return.
 - New private DFS helpers `dfs_bb` and the `par` variant that thread a running `n_neg_in_path` integer through the recursion and apply the UB-vs-threshold cut at each extension step (after the existing `nxt < start`, `visited`, BFS-distance, and `pruner.extend_ok` checks — ABB is the last cheap rejection before the recursive call).
- `hymeko_graph/tests/abb_global_topk.rs` — new integration test file: parity vs the non-ABB path on a synthetic Erdős–Rényi fixture (with and without balance pruner), plus an admissibility property test.

- `hymeko_graph/benches/abb_global_topk.rs` — new Criterion benchmark: 5-iteration warm-up + 5-iteration measure on a deterministic synthetic graph at $k=4$, $K=10\,000$, asserting post-fix median $\leq 0.70\times$ pre-fix median (a conservative target; probe predicts $5\text{--}10\times$).
- `hymeko_graph/examples/profile_topk_balance.rs` (already exists from the prior task) — a new sibling `profile_topk_balance_bb.rs` added to drive the flamegraph capture for the post-fix profile.

CORE.YAML items touched

None. `hymeko_graph` is not in `crates/files/globs`; `tests/**`, `benches/**`, and `examples/**` are in `allowlist`. `criterion` is the canonical Rust benchmarking tool per `CLAUDE.md §10` and is already a dev-dependency.

Interface changes

New public trait

```
/// Score function paired with an admissible upper bound on the
/// closed cycle's score given a partial path. ABB uses the bound
/// to prune branches whose best possible cycle score cannot beat
/// the heap's current minimum.
pub trait BoundedScorer: Sync + Send {
    /// Score a closed cycle. Same contract as the existing
    /// 'Fn(&[u32], &[i8]) -> f64' scorers.
    fn score(&self, vs: &[u32], signs: &[i8]) -> f64;

    /// Upper bound on the closed cycle's score given:
    /// - 'n_neg_so_far': count of negative edges already in
    ///   the partial path (the path has 'k_len - k_remaining'
    ///   edges so far).
    /// - 'k_remaining': number of edges still to add (closing
    ///   edge inclusive).
    /// - 'k_len': total cycle length.
    ///
    /// # Admissibility postcondition
    /// For every closed cycle reachable from the partial state,
    /// 'self.score(closed) <= self.upper_bound(...)'.
    /// Implementations must satisfy this; tests in
    /// 'tests/abb_global_topk.rs::ub_admissible_*' enforce it on
    /// random fixtures.
    fn upper_bound(&self,
                   n_neg_so_far: usize,
                   k_remaining: usize,
                   k_len: usize) -> f64;
}
```

Concrete impls

```
pub struct FractionNegativeScorer;
impl BoundedScorer for FractionNegativeScorer {
    fn score(&self, _vs: &[u32], signs: &[i8]) -> f64 {
        if signs.is_empty() { return 0.0; }
        let n_neg = signs.iter().filter(|&s| s < 0).count() as f64;
        n_neg / signs.len() as f64
    }
}
```

```

    }
    fn upper_bound(&self, n_neg_so_far: usize, k_remaining: usize,
                  k_len: usize) -> f64 {
        // Best case: every remaining edge is negative.
        let total_neg = n_neg_so_far + k_remaining;
        total_neg as f64 / k_len as f64
    }
}

pub struct BalanceScorer;           // UB = 1.0 (sign product is +/- 1)
pub struct SignProductAbsScorer;    // UB = 1.0
pub struct LowRootScorer;          // UB = 0.0 (nondecreasing in start)

```

New entry points

```

pub fn enumerate_top_k_cycles_bb<P, S>(
    graph: &SignedGraph, k_len: usize, pruner: &P,
    k_keep: usize, scorer: &S,
) -> Vec<TopKCycle>
where
    P: CyclePruner,
    S: BoundedScorer;

pub fn enumerate_top_k_cycles_par_bb<P, S>(
    graph: &SignedGraph, k_len: usize, pruner: &P,
    k_keep: usize, scorer: &S,
) -> Vec<TopKCycle>
where
    P: CyclePruner + Sync,
    S: BoundedScorer;

```

ABB descent (private)

```

// at each extension step, after existing checks pass:
let nxt_sign = signs_csr[s + pos];           // CSR-aligned, no HashMap
let new_n_neg = n_neg + ((nxt_sign < 0) as usize);
let k_remaining = k_len - (path.len() + 1); // edges still to add
let ub = scorer.upper_bound(new_n_neg, k_remaining, k_len);
let threshold = if heap.len() == k_keep {
    heap.peek().map(|e| e.score).unwrap_or(f64::NEG_INFINITY)
} else { f64::NEG_INFINITY }; // not full -> no threshold
if ub <= threshold {
    continue; // branch is provably dominated
}
path.push(nxt);
visited[nxt as usize] = true;
dfs_bb(start, ..., heap, &dist, scorer, new_n_neg);
path.pop();
visited[nxt as usize] = false;

```

The local heap threshold is per-rayon-fold-task (each task accumulates into its own heap; the reduce step merges). This is a safe but *loose* bound: a thread whose heap is still filling will not benefit from a peer thread's higher threshold. A cross-thread atomic threshold could help further; *deferred* as a follow-up optimisation pending v1 measurements.

Test strategy

Unit (src/topk_cycles.rs::tests + property tests)

- **Admissibility on hand-checked fixtures:** for each `BoundedScorer`, on a 6-vertex deterministic signed graph, enumerate every $k=4$ cycle and assert $\text{score}(c) \leq \text{UB}(0, k, k)$ (degenerate root call) and $\text{UB}(\text{full path}, 0, k) = \text{score}(c)$ (terminal call).
- **UB monotonicity:** for fixed k_{len} , $\text{UB}(n', k_{\text{rem}} - 1, k_{\text{len}})$ where $n' \in \{n, n + 1\}$ (one more edge committed) is at most $\text{UB}(n, k_{\text{rem}}, k_{\text{len}})$. A *loosening* of the UB across descent is a contract bug; this catches it.

Integration (tests/abb_global_topk.rs)

- **Parity vs non-ABB** on a 200-vertex Erdős–Rényi signed graph (LCG seed 42; matches the existing `tests/csr_sign_lookup.rs` fixture style): `enumerate_top_k_cycles_par_bb`'s output is the same *multiset of (canonical cycle, score)* as `enumerate_top_k_cycles_par`'s (modulo score-tie tie-breaking, asserted at $\geq 95\%$ canonical-cycle agreement same as the existing tests).
- **Both pruners** (NoOp and CartwrightHarary OnlyBalanced).
- **Admissibility on random fixtures:** 1000 random partial-path states, assert UB is at least as large as every feasible completion's score. (Property-style; run for each `BoundedScorer`.)

Performance (Criterion)

Per CLAUDE.md §3 benchmark stability rule (5-iteration warm-up + 5-iteration measure, report median + IQR + worst):

- **Bench input:** 5 000-vertex Erdős–Rényi signed graph (deterministic LCG seed, fixture-hashed at the start of each run; same seed every iteration).
- **Configs:** $k=4$, $K \in \{1\,000, 10\,000, 100\,000\}$, `pruner = CartwrightHararyPruner(OnlyBalanced)`, `scorer = FractionNegativeScorer`.
- **Numerical budget:** post-fix median $\leq 0.70 \times$ pre-fix median at $K=10\,000$. Asserted in a Criterion regression block; failure aborts the bench.
- **Stretch goal:** post-fix median $\leq 0.30 \times$ pre-fix median at $K=10\,000$ on the Epinions-shaped synthetic, matching the probe's $5 \times$ prediction.

Pre-/post-fix flamegraph

Both flamegraphs (low-frequency, $-F\,99$ to keep sample loss below 30% as established by the prior task) attached to the report. The post-fix flamegraph must show `dfs_bb`'s total cycle share *below* the prior `dfs`'s share by at least the proportion of branches pruned, with attribution shifted to BFS-distance pruning or to the heap min check (acceptable; that's where the work moves).

Performance budget

Metric	Pre (probe)	Post (budget)	
Epinions $k=4$, global $K=10\,000$, balance pruner	probe-tbd	—	
5 000-v ER $k=4$, $K=10\,000$, balance pruner	Criterion baseline	$\leq 0.70\times$ baseline	
5 000-v ER stretch goal	Criterion baseline	$\leq 0.30\times$ baseline	
Output cycle-set agreement vs non-ABB	—	$\geq 95\%$ canonical	
Peak RSS during enumeration	—	no regression vs no-ABB	informal —
16 GB hard cap (§4)	comfortably under	comfortably under	

Rollback path

The change is one Rust file (`topk_cycles.rs`), additive only — no signature change to existing entry points, no behavior change to existing callers. Single revert removes:

- BoundedScorer trait + 4 concrete impls
- `enumerate_top_k_cycles_bb` + `enumerate_top_k_cycles_par_bb` + private DFS helpers

```
git revert <commit-sha>
cargo test --release # all green pre-change (existing entry points
    untouched)
```

Halt condition: if the post-fix flamegraph shows `dfs_bb` share $\geq$ the no-ABB `dfs` share (i.e., ABB adds work without pruning), revert and reopen the plan. Per CLAUDE.md §10 “a measurement contradicts an assumption in the plan”.

Architecture diagram

See `plan.tikz` (rendered below) for the ABB descent state machine. See `plan.mmd` for the call-sequence diagram.

